SlimCDDevice Developer Guide

This device library provides functionality that allows to dynamically discover and instantiate PIN pad libraries API. The PIN pad libraries discovery is performed using a reflection, the library scans assemblies using a meta-context without fully loading them, trying to locate the `ISlimCDDeviceAsync` interface implementation. It also provides functionality for instantiating the PIN pad libraries and the instance lifecycle management. The library can be used on a different platforms (Tested on Windows, Android and iOS).

Using the Library

Here are the steps for using the library:

1. Optionally, you can discover device assemblies by calling:
   `await Assemblies.DiscoverAsync(nameof(ISlimCDDeviceAsync), Cts.Token);`
   It will return a response which in a case of success will contain a collection of the device assembly description objects (`IAssemblyInfo`) that can be used to load those assemblies.
2. Load the selected device assembly by calling:
   `await Assemblies.GetInstance(selectedDeviceAssembly, PlatformSpecific, Cts.Token);`
   If successful it will return the device library instance object of the type `ISlimCDDeviceAsync`.

After you finished using the device library object (`ISlimCDDeviceAsync`) you need to properly dispose it, the object does implement `IAsyncDisposable` and `IDisposable` interfaces. The disposal will cause the library to disconnect from the device and it will also dispose the internal resources.
If disconnection was performed in a separate call, there is no need to dispose the object, **however**, if `Disconnect` was not called it is required to call `Dispose` to avoid memory leaks.

Library Methods

Discovering Device Assemblies

```
async Task<IResponse<IReadOnlyCollection<IAssemblyInfo>>> DiscoverAsync(
          string interfaceName,
          CancellationToken ctx = default);

async Task<IResponse<IReadOnlyCollection<IAssemblyInfo>>> DiscoverUsingPath(
          string interfaceName,
          string path,
          CancellationToken ctx = default,
          string searchPattern = DEVICE_LIBRARY_SEARCH_PATTERN,
          SearchOption searchOption = SearchOption.AllDirectories);
```

Those methods can be used to discover device assemblies. The call to it is optional, as an alternative you can implement `IAssemblyInfo`, instantiate it with the assembly data and use it for obtaining the PIN pad assembly API instance, completely avoiding the use of `DiscoverAsync`.
To discover PIN pad assemblies those methods first check the assemblies that has been already loaded to the process, then they check the assemblies in the same path where the `SlimCDDevice` library is located, or in the case of the second method in the path that has been provided in the method call. `SlimCDDevice` library tries to find an implementation of `ISlimCDDeviceAsync` PIN pad API and if found returns it in the response collection.
The library is IIS aware and it will search the original files location, not the temp folder if run under IIS.

When searching for the PIN pad libraries located in the storage, to be able to do that the libraries dependencies also need to be present in the path to avoid those dependencies to be excluded there is a static list property `Assemblies.RequiredAssemblies` where those dependencies can be added. By default it already contains the common dependencies list for all platforms, so no changes needed unless a new device library with a new dependency is added (Or a new platform support).
Another static list property - `Assemblies.ExcludeAssemblies` contains assemblies that should be excluded from the search (the one that we for sure know that are not the device assemblies). You may add your project assemblies into this list like that:
If you have "`MyLibrary.dll`", add it like `Assemblies.ExcludeAssemblies.Add("MyLibrary");`
Excluding assemblies from discovery is optional, but when it is done is accelerates the discovery process (Less libraries to check, less unavoidable handled exceptions in a case of an unsupported assembly formats). Add this code to exclude your payment app from the discovery process:

```
Assemblies.ExcludeAssemblies.Add(MethodBase.GetCurrentMethod()!.DeclaringType!.Ass
embly.GetName().Name);
```

Parameters

`interfaceName`	To locate the PIN pad device assemblies this value should always be a string of: `nameof(ISlimCDDeviceAsync)`
`path`	The relative or absolute path to the directory to search. This string is case-insensitive. It is platform specific.
`ctx`	Cancellation token for cancelling the device discovery.
`searchPattern`	The search string to match against the names of files in path. This parameter can contain a combination of valid literal path and wildcard (* and ?) characters. It doesn't support regular expressions.

`searchOption`	Specifies whether the search operation should include only the current directory or should include all subdirectories. The values can be:
	`SearchOption.AllDirectories`
	`SearchOption.TopDirectoryOnly`

Returns

`IResponse< IReadOnlyCollection< IAssemblyInfo>>`	To determine of the request was successful or if it is failed you can use the following properties:
	`IResponse<>.IsSuccess`
	`IResponse<>.IsFailure`
	
	If the request was successful then the data property:
	`IResponse<>.Data`
	
	Will contain the collection of the device assemblies describing objects of the type:
	`IReadOnlyCollection<IAssemblyInfo>`
	
	If the request has failed then the returned collection will be null and the error message will be returned in:
	`IResponse<>.Message`

Exceptions

`OperationCanceledException`	The discovery has been cancelled by using the cancellation token.

All other exceptions are intercepted and returned as errors.

Note. PIN pad assemblies discovery process is platform dependent, in a case of iOS any and Android Release build there is no access to the assemblies located on the storage, so only the assemblies that has been loaded into the process will be checked. And due to the dynamic nature of the PIN pad assemblies discovery and instantiation there are additional steps that need to be performed:

1. Set the linking to be done to SDK only.
2. Even if the linking of the user libraries is disabled, if the compiler doesn't find a direct assembly reference it will discard those assemblies. To trick the compiler to include the required assemblies add the following code to `MainActivity` on Android and `Main` on iOS:

```
_ = typeof(IDTechNeo.IDTechNeoHal).FullName;
_ = typeof(IngenicoUpp.IngenicoUppHal).FullName;
```

Loading Device Assemblies

```
async Task<IResponse<ISlimCDDeviceAsync>> GetInstance(
            IAssemblyInfo deviceAssembly,
            IPlatformSpecific platformSpecific = null,
            CancellationToken ctx = default);
```

The method loads the specified device assembly and all the dependencies into the process returning back the assembly interface - `ISlimCDDeviceAsync`. After the instance has been created it is stored internally so if the call is repeated the existing instance will be returned. If the call is repeated for a different assembly then the stored instance will be disposed and the new one created instead, the previous assembly stays loaded into the process.

Parameters

<table>
<tr><td>deviceAssembly</td><td>Object of the type <code>IAssemblyInfo</code> which describes the assembly that we have to load. The objects of this type are returned by the assembly discovery methods, but you are not limited to using just them. Create a class and derive it from <code>IAssemblyInfo</code> in this class the only properties that are used in the call to <code>GetInstance</code> are:
<table>
<tr><td>Location</td><td>Location of the assembly file. If the file doesn't exist then the method will try to load the assembly using <code>Name</code>.</td></tr>
<tr><td>Name</td><td>The name of the assembly. The search will be done in the assemblies already loaded into the process.</td></tr>
</table></td></tr>
<tr><td>platformSpecific</td><td>Contains the platform specific APIs that can be used by the device libraries, like main thread invoker, initializer, and a custom communication manager. The call to <code>GetInstance</code> does a dependency injection of this object into the device assembly object. Although this parameter is optional it is strongly required to provide it, otherwise some functionality may be not accessible or work correctly on a different platforms. See the examples of the code in the <code>HardwareSample</code> app.</td></tr>
<tr><td>ctx</td><td>Cancellation token for cancelling the loading.</td></tr>
</table>

Returns

`IResponse<ISlimCDDeviceAsync>`	To determine of the request was successful or if it is failed you can use the following properties:

	`IResponse<>.IsSuccess`
	`IResponse<>.IsFailure`
	
	If the request was successful then the data property:
	`IResponse<>.Data`
	
	Will contain the device assembly object of the type:
	`ISlimCDDeviceAsync`
	
	If the request has failed then the returned data object will be null and the error message will be returned in:
	`IResponse<>.Message`

Exceptions

`OperationCanceledException`	The discovery has been cancelled by using the cancellation token.

All other exceptions are intercepted and returned as errors.

Note. You may also notice that the library exposes the method `CreateInstance`. This method just creates an instance of an assembly, and because device assemblies require additional initialization steps the method `CreateInstance` should not be used directly.

Additional Considerations

1. Add the following dependencies to your project:

`SlimCDTypeLib`	The library contains the type definitions shared by all other SlimCD libraries. No dependencies.
`SlimCD`	Contains functionality for interacting with SlimCD gateway. No dependencies.
`SlimCDDevice`	The library described in this document. Manages the device specific libraries. Depends on `SlimCDTypeLib`.
`DeviceFramework`	Implements functionality common to all payment devices, it is used as a base for building a device specific libraries. Depends on `SlimCDTypeLib`, `SlimCD`.
`IngenicoUpp`	Ingenico specific device library. All platforms. Depends on DeviceFramework.
`CommManagerAndroid`	Communication module plugin for DeviceFramework that enables it to connect to devices using Bluetooth on Android platform. Currently used only with Ingenico devices. Depends on `DeviceFramework`, `SlimCDTypeLib`, `SlimCD`.
`IDTechNeo.Standard`	IDTech specific device library. This is a .NET Standard 2.1 (And other) library that should be used in non-MAUI/Xamarin projects. Depends on `DeviceFramework`.
`IDTechNeo.Maui`	IDTech specific device library. This library should be used in MAUI projects. Depends on `DeviceFramework`.
`IDTechNeo.Xamarin`	IDTech specific device library. This library should be used in Xamarin projects. Depends on `DeviceFramework`.

2. Android. If you need to connect to **Ingenico** PIN pads using Bluetooth you need:

1. Add the dependency package `CommManagerAndroid`.
2. If you are using `PlatformSpecific` implementation provided in `HardwareSample`, in the `MainActivity`, in the beginning of `OnCreate` method override add this code:

```
PlatformSpecific.AndroidContext = this;
```

3. **Android.** If you are planning to support **IDTech** devices, in `MainActivity`, in the beginning of `OnCreate` add **one** of:

```
IDTech.Maui.Comm.IDTechBinding.Init(this); // If IDTechNeo.Maui
Xamarin.IDTech.AndroidBinding.IDTechBinding.Init(this); // If IDTechNeo.Xamarin
```

And at the end of the method, if you plan to connect to IDTech devices using USB:

```
IDTech.Maui.Comm.IDTechBinding.enableUSB(); // If IDTechNeo.Maui
Xamarin.IDTech.AndroidBinding.IDTechBinding.enableUSB(); // If IDTechNeo.Xamarin
```

4. **Android.** The following Permissions are required to be declared in Android app:

`ACCESS_NETWORK_STATE` `INTERNET`	Required to be able to access Internet.
`ACCESS_COARSE_LOCATION` `ACCESS_FINE_LOCATION`	Required by IDTech libraries.

`BLUETOOTH`
`BLUETOOTH_ADMIN`
`BLUETOOTH_CONNECT`
`BLUETOOTH_SCAN`
	Required only if you need to access devices using Bluetooth.

`MODIFY_AUDIO_SETTINGS` `RECORD_AUDIO`	Required only if you need to access IDTech devices using Audio Jack connector.
`READ_EXTERNAL_STORAGE`	Required by `SlimCDDevice` in a case if need to discover/load assemblies by using a specific path.

5. **iOS.** If you are planning to support IDTech devices, in the same place add **one** of:

```
IDTech.Maui.Comm.IDTechBinding.Init(this); // If IDTechNeo.Maui
Xamarin.IDTech.iOSBinding.IDTechBinding.Init(); // If IDTechNeo.Xamarin
```

6. **iOS.** The following needs to be added to `plist` in the iOS app:

`<key>UISupportedExternalAccessoryProtocols</key>`
`<array>`
`  <string>com.idtechproducts.neo</string>`
`  <string>com.idtechproducts.reader</string>`
`  <string>com.idtechproducts.BTPay 200</string>`
`  <string>com.idtechproducts.BTMag</string>`
`</array>`
	This needs to be added if you are planning to use any of the following IDTech readers:
	com.idtechproducts.neo - VP3350
	com.idtechproducts.BTMag - BTMag
	com.idtechproducts.BTPay 200 - BTPay
	com.idtechproducts.reader - iMag

`<key>NSBluetoothAlwaysUsageDescription</key>`
`<string>Captures Payment Information</string>`
`<key>NSBluetoothPeripheralUsageDescription</key>`
`<string>Captures Payment Information</string>`
	This needs to be added if you are planning to connect to IDTech devices using Bluetooth.

`<key>NSMicrophoneUsageDescription</key>` `<string>Captures Payment Information</string>`	This needs to be added if you are planning to connect to IDTech devices using Audio Jack.